

MEXORA RH INTELLIGENCE

Data Lake Architecture Design Document

Architecture & Design Justification

1. Project Context

Mexora is an e-commerce company headquartered in Tanger, Morocco. Its HR Director plans to recruit 5 data profiles — Data Engineers, Data Analysts, and a Data Scientist — to support the company's digital expansion. Before launching recruitment, Mexora commissioned a market intelligence study of the Moroccan IT job market.

This document describes the architecture of the Data Lake built to support that study: ingestion of 5,000 IT job offers from three sources (Rekrute, MarocAnnonce, LinkedIn Morocco), transformation through three quality zones (Bronze, Silver, Gold), NLP-based skill extraction, and DuckDB-powered analytics.

2. Architecture Overview — Bronze / Silver / Gold

The Data Lake follows a medallion architecture with three zones of increasing data quality and business readiness. This pattern separates concerns clearly: raw preservation (Bronze), quality and standardisation (Silver), and business aggregation (Gold).

Zone	Purpose	Format	Contents
Bronze	Immutable raw archive	JSON (partitioned)	5,000 raw offers partitioned by source/month
Silver	Cleaned, standardised	Parquet (Snappy)	offres_clean.parquet, competences.parquet
Gold	Business aggregations	Parquet (Snappy)	5 analytical tables ready for DuckDB / BI tools

3. Design Justification — Four Key Questions

Zone	Format	Why this format?	Why not others?
Bronze	JSON	Source data arrives as JSON from scraping. Storing it unchanged ensures zero transformation loss at ingestion. JSON is self-describing and human-readable for debugging. Bronze is never queried analytically so query speed does not matter here.	CSV would break nested fields like <code>langue_requise</code> . Parquet requires a fixed schema at write time — impossible when raw data has inconsistent types and missing fields across sources.
Silver	Parquet (Snappy)	After cleaning the schema is stable and typed. Parquet is columnar — DuckDB reads only the columns needed per query. Snappy compression reduces file size by ~85% vs raw JSON while staying fast to decompress.	CSV has no type system and handles embedded commas in description fields poorly. JSON is row-based, which is inefficient for column-focused analytics.
Gold	Parquet (Snappy)	Gold tables are pre-aggregated and consumed by DuckDB, Power BI, and Metabase. Parquet's columnar layout means a salary query reads only 2 columns out of 20. It is universally supported by all BI tools.	A relational database like PostgreSQL would require a running server, defeating the local/serverless approach. Excel cannot handle columnar compression or typed nulls reliably.

Question 1: Why preserve raw data in Bronze unchanged?

Bronze is the permanent, faithful archive of exactly what was ingested. If a cleaning rule in Silver contains a bug — for example a salary regex that incorrectly nullifies valid values — the Silver pipeline can be re-run from Bronze without re-scraping the source websites. This is called idempotent reprocessing. Without Bronze, a faulty transformation corrupts the only copy of the data with no recovery path. It also guarantees full data lineage: auditors can always verify where a number came from.

Risks if we skip Bronze:

- Irreversible data loss — a bad transform discards fields permanently.
- No lineage — impossible to verify how a final figure was derived.
- Pipeline brittleness — one bad Silver run destroys everything.

Question 2: What is schema-on-read vs schema-on-write?

Schema-on-write (used in the DWH of mini-project 1) enforces a fixed schema at ingestion — any data that does not conform is rejected. Schema-on-read (used in this Data Lake) applies no schema at ingestion. Any JSON structure is accepted and stored as-is. The schema is only applied later when the data is read and processed in Silver.

This is the right approach here because Rekrute, MarocAnnonce, and LinkedIn may deliver slightly different field structures. We want to store everything first and handle differences explicitly during cleaning, rather than rejecting records at ingestion and losing data.

Question 3: How is partitioning defined in each zone?

Bronze is partitioned by source and by month (bronze/{source}/{YYYY_MM}/). If an anomaly appears in one source for a specific month, only that partition needs to be inspected or reprocessed without touching the rest. This also makes incremental ingestion possible in future — only new months need to be added rather than reprocessing all 5,000 offers each run.

Silver is partitioned by city and by month. The primary analytical dimension in this project is geographic (comparing Tanger vs Casablanca vs Rabat), so partitioning by city allows DuckDB to skip irrelevant partitions entirely on city-specific queries.

Gold has no partitioning — the tables are already small pre-aggregated results (a few hundred rows each). Fragmenting them would add overhead with no benefit.

Question 4: How do we prevent a Data Swamp?

A Data Swamp appears when data accumulates with no documentation, no naming conventions, and no quality enforcement — making the lake impossible to use. We prevent this through four rules:

- Consistent naming: all partitions follow {source}/{YYYY_MM}, all Parquet files have descriptive names. No ambiguous or temporary file names.
- Metadata at ingestion: Every Bronze partition file contains a header with ingestion timestamp, source, and record count — serving as a catalogue entry.
- Quality Flags: Records failing validation in the silver zone carry an anomalies column for full auditability.
- Zone discipline: Bronze is never modified after ingestion. Silver is never used for business reporting. Gold is never used for reprocessing. Each zone has one job and one job only.

4. Architecture Diagram

The diagram below illustrates the full data flow from source ingestion through the three zones to end consumers. It was produced using draw.io and shows the formats, tools, and access levels at each stage. The Medallion architecture ensures data integrity by moving from raw storage (Bronze) to refined, analytical tables (Gold) tailored for Mexora's strategic recruitment goals. Data Engineers manage the pipeline health (Bronze/Silver), while Data Scientists use the cleaned Silver data for predictive modeling. Data Analysts and the HR Director consume the final Gold aggregates to extract actionable insights and track Moroccan market KPIs through DuckDB and dashboards. This design enforces rigorous data governance while empowering each role with the specific level of data granularity they require.

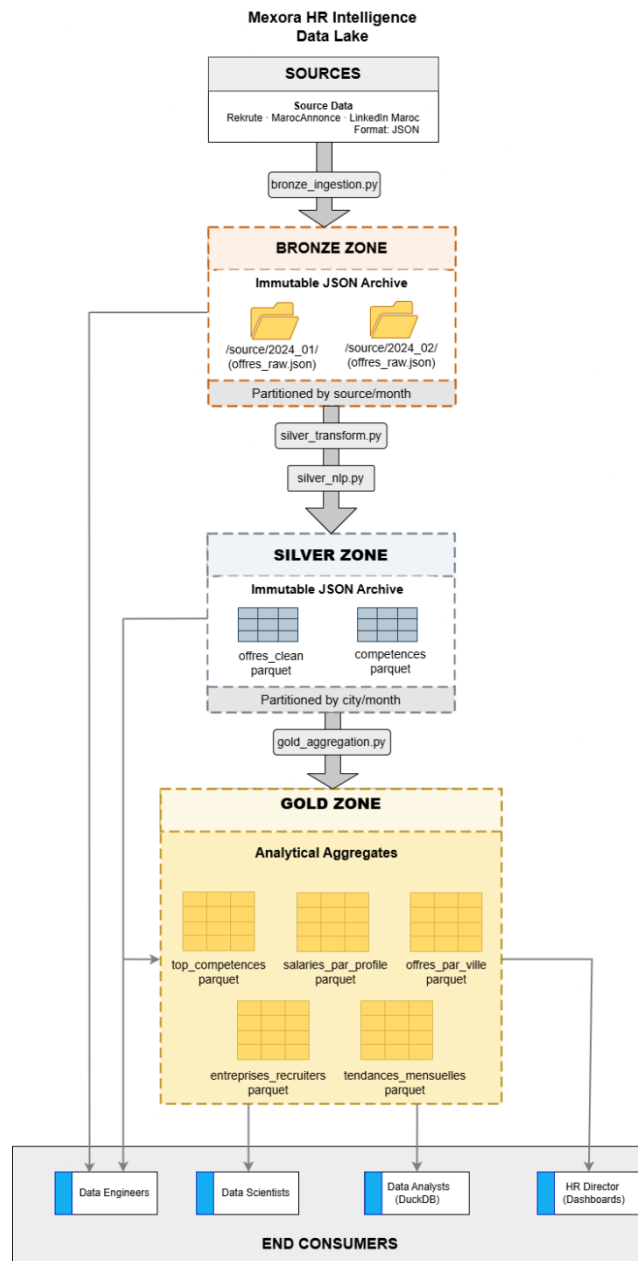


Figure 1 — Mexora RH Intelligence Data Lake Architecture (Bronze / Silver / Gold)